
Part IV

Artificial Neural Networks and Deep Learning

From the Perceptron to Artificial Neural Networks

17

A single neuron in the brain is an incredibly complex machine that even today we don't understand. A single 'neuron' in a neural network is an incredibly simple mathematical function that captures a minuscule fraction of the complexity of a biological neuron.

Andrew Ng (born 1976), American computer scientist and technology entrepreneur

17.1 A First Model of a Neuron

The biological neuron is often used to explain the motivation behind the computational model(s) of neurons. While, in the early days of the computational neurons, it was certainly true that they were designed to mimic biological neurons, hoping to create an “artificial intelligence”. However, later on, with the emergence of more powerful models this was changed, and it is no longer the goal to imitate the biological neuron (unless we are diving into “neuromorphic computing”). In the following, we trace the historic development *artificial neural networks* (ANNs) and thereby understand how, why, and when these single neuron models became the “launch pad” for today’s powerful and complex deep learning architectures.

17.1.1 The Biological Neuron

The name *neuron* was coined in the late nineteenth century by the anatomist HEINRICH WILHELM WALDEYER [11], who already speculated 1881 that biological neurons might be the basic building blocks of the nervous system. Generally, neurons are nerve cells for processing and transmitting chemical and electrical

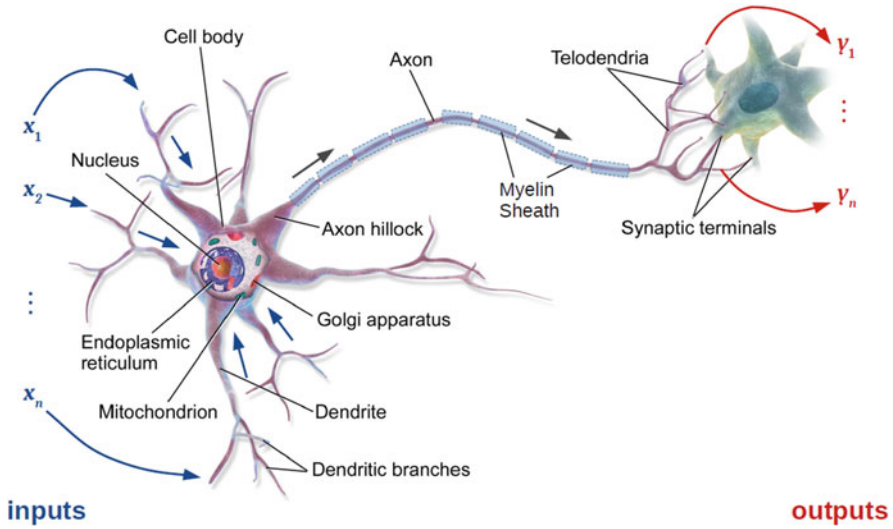


Fig. 17.1 Anatomy of a multipolar neuron: left is the presynaptic cell, right is the postsynaptic cell. The input signal arrives at dendrites/dendritic branches, is processed in the soma/cell body, which passes the signal through the axon to the synapses or other neuron's dendrite (indicated by the red arrows). (Image adapted from [2], licensed under CC BY 3.0 license¹)

signals. Typically, a large number of neighboring neurons (even up to 200,000) are interconnected with each other and thereby exchanging information.

A typical neuron consists of a cell body, the *soma*, whose purpose is to process the received signals. The links between the neurons consist of *dendrites* (from the Greek δένδρον, pronounced dendron = “tree”) and the *axon* (from the Greek ἄξων, pronounced axōn = “axis”): through the dendrites, the signal is received from other neurons, and the *axon* transmits the output. The *synapse* is located at the end of the axon and acts as connection point between two neurons; mostly, it “translates” electrical signals into chemical signals, the so-called *neurotransmitters*. A particular type with one axon and two or more dendrites/dendritic branches is called *multipolar neuron*, see the sketch in Fig. 17.1. Neurons are most commonly encountered in the central nervous system and spinal cord of almost all animals.

Neurons react to electrical signals, or more precisely to the change of voltage over time. If this change within a small time span is sufficiently large, then the neuron “fires” by emitting an electrochemical pulse. This pulse is transmitted by the axon, getting weaker with increasing distance, and eventually reaches the synaptic connection where it causes an excitatory or inhibitory response. The particular kind of response can be modified in a number of different ways. Hence, the signal transfer can be adapted by certain types of stimuli—a process that is called

¹ <https://creativecommons.org/licenses/by/3.0/deed.en>

“synaptic plasticity”.² It was only about 10 years after the neuron got its name from H. W. WALDEYER that EUGENIO TANZI, an Italian neuropsychiatrist, hypothesized that during repetitions of a learning process, a repeated neuronal activity was initiated that reinforces the current synaptic connections and makes the crossing of the signal easier [1]. Further information on the biological process from the point of view of computer science can be found in, e.g., [6].

17.1.2 Introducing...the McCulloch-Pitts Model

Two persons are responsible for the *McCulloch-Pitts* (MCP) model, which is named after them: WARREN. S. MCCULLOCH and WALTER PITTS. The MCP model [3] was developed in 1943 and is also called *Threshold Logic Unit* (TLU) model. It was the first computational model designed to resemble or imitate a biological neuron. Its main goal was to make a decision $y \in \{0, 1\}$, i.e., to answer a question with “yes” or “no” based on some input $x_i \in \{0, 1\}$. There, the values 0 and 1 typically correspond to the Boolean values `False` and `True`.

Example 17.1 (A decision that the MCP model could make)



A decision made by the MCP model might be answering the question “Is it going to rain in the afternoon?”. The decision could be based on the input information x_1 : “The sun is shining right now” (“yes” or “no”); x_2 : “The air humidity is above 70%” (“yes” or “no”). The output is then $y = 1$ (“yes, it will rain in the afternoon”) or $y = 0$ (“no, it will not rain in the afternoon”).

In analogy to the example of the biological neuron in Fig. 17.1, the flow of data would take place from left to right: the input values x_i from the left (dendritic branches) are processed or *aggregated* by the cell body and transferred to the right through the axon to the synapse, which then fires (or not). The MCP model is based on the assumption that the time for the information transfer along the axon is negligible. The aggregation of the input data x_i happens in the form of an (unweighted) sum while the fact that the synapse fires only above a certain threshold is modeled through a nonlinear *activation function* φ . Mathematically, the aggregation of the input happens in the *aggregation or transfer function* $a(\cdot)$,

$$a(\mathbf{x}) = \sum_{j=1}^n x_j, \quad (17.1)$$

² This notion is the reason why an online search for the word “plasticity” not only shows materials science results on *plastic deformation of metals* but also shows—an equally large number of—seemingly unrelated search results from the field of neuro science.

yielding the *net input* of the unit. The binary output y is then obtained by applying the activation function φ to the aggregated data

$$y = \varphi(a(\mathbf{x})) . \tag{17.2}$$

In the case of the MCP model, the activation function is taken as a unit step function with the threshold value θ ,

$$\varphi(a) = \begin{cases} 1 & \text{IF } a \geq \theta \\ 0 & \text{ELSE ,} \end{cases} \tag{17.3}$$

as depicted in Fig. 17.2a. It is important that the comparison is $\geq$ and not just $=$ as can be seen in the next section. A typical sketch of the MCP model is shown in Fig. 17.2b; Fig. 17.2c shows the commonly used short version.

Words of advice

In a number of publications and online resources, the MCP model is not always presented correctly: there, the input values are often individually weighted with factors, which, according to the publication of McCulloch and Pitts from 1943 [3], is not correct. In the original MCP model, the inputs are not individually weighted and represent Boolean values.

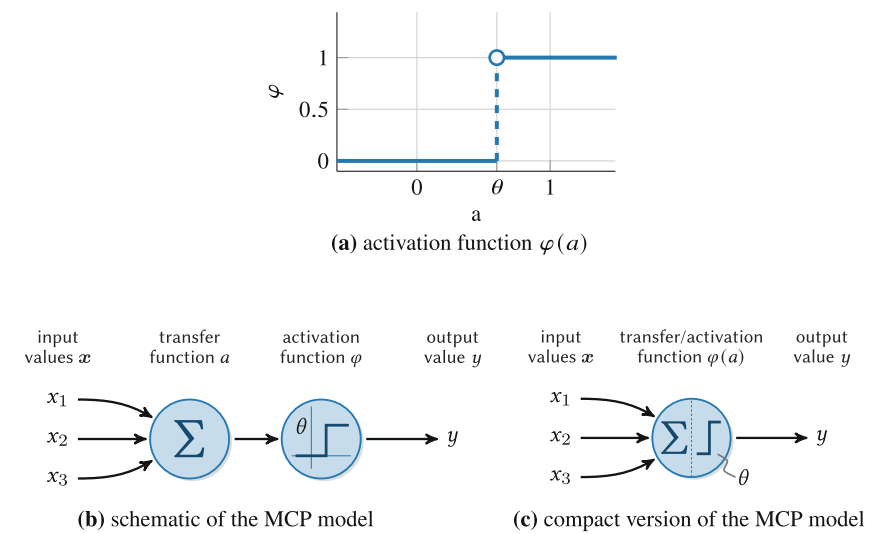


Fig. 17.2 The MCP model takes the input x_i , sums them up in the transfer function, and “makes a decision” with the activation function. For this model, the activation is the unit step function shown in (a) and has a threshold θ , which is also indicated in the detailed symbolic representation of the model in (b). (c) show a compact representations of the model for three inputs

Table 17.1 Representation of the logical AND by a MCP model with two inputs and a threshold value $\theta = 2$. The two output columns show the model output as well as the true output $x_1 \wedge x_2$

Input		Data Processing			Output
x_1	x_2	$a(\mathbf{x}) = x_1 + x_2$	$a \geq \theta ?$	$y = \varphi(a)$	$x_1 \wedge x_2$
0	0	0	$0 \geq 2 ?$	0	0
1	0	1	$1 \geq 2 ?$	0	0
0	1	1	$1 \geq 2 ?$	0	0
1	1	2	$2 \geq 2 ?$	1	1

17.1.3 Boolean Decision Problems

Many problems can be turned into a *Boolean decision problem*, i.e., decisions where the input data and the outcome (i.e., the decision) can be reduced to statements with “yes” or “no” answers. In other words, the output data are *categorical data* and thereby solve binary classification problems as there are only two possible outcomes (0 or 1), which, however, can be interpreted, e.g., as “the testing specimen shows failure” or “the testing specimen is intact.”

In the MCP model, both the input values x_i and the output y can only take the values 0 or 1, which can then be interpreted as the Boolean values $\text{True} \equiv 1$ and $\text{False} \equiv 0$. Therefore, the model has the potential to solve certain Boolean decision problems, hence the alternative name “Threshold Logic Unit” (TLU).

We will now investigate if the MCP model would be able to represent the fundamental Boolean operations AND, OR, and NOT, and thus if the model would be able to make such a decision. A numerical, automated strategy for finding the most suitable value of θ is a different task and will be addressed for subsequent, more complex models below.

• Logical AND operation

This logical operator is represented using the $\wedge$ symbol. For example, for two input values x_1 and x_2 , it is $x_1 \wedge x_2$. The result of the logical AND is True if and only if all input values are True . For a neuron with two inputs, this is the case if the threshold value is chosen as $\theta = 2$; for a logical unit with n inputs, θ must be set to n .

This can be easily seen in Table 17.1 by comparing the column “ $a = x_1 + x_2$ ” with the column “ $x_1 \wedge x_2$ ”: only when a changes from 1 to 2 the output should change from 0 to 1 and thus $\theta = 2$. Thus, the MCP model is able to represent the logical AND function.

• Logical OR operation

This logical operator is represented using the $\vee$ symbol. For example, for two input values x_1 and x_2 , the OR function is $x_1 \vee x_2$. The result of the logical OR is True if at least one of the input values is True . For a neuron with two and more inputs, this is the case if the threshold value is chosen as $\theta = 1$ as shown in Table 17.2.

Table 17.2 Representation of the logical OR by a MCP model with two inputs and a threshold value $\theta = 1$. The two output columns show the model output as well as the true output $x_1 \vee x_2$

Input		Data Processing		Output
x_1	x_2	$a(\mathbf{x}) = x_1 + x_2$	$a \geq \theta ?$	$y = \varphi(a)$
0	0	0	$0 \geq 1 ?$	0
1	0	1	$1 \geq 1 ?$	1
0	1	1	$1 \geq 1 ?$	1
1	1	2	$2 \geq 1 ?$	1

Thus, the MCP model is able to represent the logical OR function.

- *Logical NOT operation*

This logical operator is represented using the $\neg$ symbol. For example, for an input x , the NOT function is $\neg x$. The result of the logical NOT function is True if the input is False (and vica versa). The NOT function is the counterpart to an *inhibitory synapse*.

For representing the NOT function, though, it would be necessary to weight the input with -1 and adjusting threshold alone is not sufficient, which can be shown as follows: Assume that $x = 0$. It follows:

$$x = 0 \Rightarrow y = \neg x = 1 \quad (17.4)$$

As for a single input, the transfer function becomes the identity function, $a(x) = x$, we are looking for the θ that fulfills the inequality

$$a(x) = x = 0 \geq \theta \Rightarrow \theta \leq 0. \quad (17.5)$$

Assuming $x = 1$ it is $y = 0$, and we are looking for values of θ that *do not* satisfy the inequality, $a = x \geq \theta$, such that y evaluates to 0 (False):

$$x \not\geq \theta \Rightarrow \theta > 1. \quad (17.6)$$

As it is impossible to fulfill Eqs. 17.5 and 17.6, the NOT operation cannot be represented by the MCP model.

Another Boolean operator is the XOR operator, which is represented by the symbol $\oplus$ or also by $\dot{\vee}$. Its result is True if and only if $x_1 \neq x_2$, i.e., one input is True while the other one is False. The XOR operator can be replaced by a combination of AND and OR operations together with the NOT operator. As the MCP model cannot represent the NOT operator it is also not able to represent the XOR operator. Thus, a more flexible model is required that, as it turns out, needs to at least be able to “weight” the inputs with individual and adaptable factors.

Things to Remember about the McCulloch-Pitts (MCP) model:

- The MCP (or TLU) model is the simplest possible model motivated by the biological neuron.
- The input data have the values 1 (True) or 0 (False) and are aggregated (summed up); ...
- ...if this value is larger than a threshold value, the output is 1 (or True); otherwise, it is 0 (or False).
- The model can only make the Boolean decisions “AND” and “OR” while operations “NOT” and “XOR” cannot be represented by the model.

17.2 The Rosenblatt Perceptron

After the pioneering development of the McCulloch-Pitts model, it took another 15 years until the *perceptron* as an enhancement of the MCP model was developed by the psychologist Frank Rosenblatt: in 1957, he published the report “The Perceptron: a perceiving and recognizing automaton,” whose aim he specified as to “[...] *formulate a brain analogue useful in analysis*” [7]. In this publication, the word “perceptron” originally denoted an interconnected network *structure* and not, as it is typically understood today, the individual unit. The first hardware implementation was the Mark I perceptron, which was assembled in 1957 at Cornell Aeronautical Laboratory in Buffalo, New York. It was a pure hardware realization of the perceptron “network” where weights were represented as potentiometers, which were adjusted by electro-motors. A short overview about the work of Rosenblatt can be found in [5].

The name *perceptron* was derived from the fact that the inputs consisted of 20×20 cadmium sulfide photo cells for “perceiving” a 400 pixel image; the output consisted of signal lights that could be activated when a threshold—similar to the MCP model—was exceeded. Activation was mutually exclusive such that two signals would not be activated at the same time. Further details can be found in the original publication [7].

In the following, we introduce one of such units and will subsequently denote this as a *perceptron*; if more than one of these units are combined, then we will denote this as a network of perceptrons.

17.2.1 The Mathematical Model of the Rosenblatt Perceptron

The first main difference in the MCP model is that the inputs x are weighted with individual values. The second difference is that the input values can take real values while for the MCP they could only be Boolean, i.e., 0 or 1. Additionally, a *bias* acting as offset for the weighted and aggregated inputs was introduced. With this

the transfer function takes the shape of a linear combination,

$$a(\mathbf{x}) = b + \sum_{j=1}^n w_j x_j, \quad (17.7)$$

where the w_j are variable weights and b is the bias. This equation can be reformulated if the bias is also interpreted as weight such that the new input and weight vector looks

$$\tilde{\mathbf{x}} = [1, x_1, \dots, x_n]^T \quad \text{and} \quad \tilde{\mathbf{w}} = [b, w_1, \dots, w_n]^T. \quad (17.8)$$

With this, the transfer function can be rewritten using the scalar product between two vectors, which is computationally efficient as already mentioned by Rosenblatt:

$$a(\mathbf{x}) = \tilde{\mathbf{w}}^T \cdot \tilde{\mathbf{x}}. \quad (17.9)$$

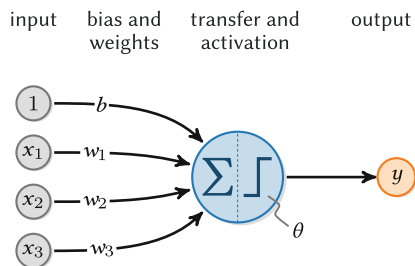
The activation function is again the unit step function with the threshold value θ (as in Eq. 17.3),

$$\varphi(a) = \begin{cases} 1 & \text{IF } a \geq \theta \\ 0 & \text{ELSE} \end{cases}. \quad (17.10)$$

A schematic representation of a perceptron with three inputs is depicted in Fig. 17.3.

Note that b does not always occur in the above equation, as it can be “brought to the other side” of Eq. 17.10, and then it just contributes to the threshold (alternative, the threshold can also be included in the bias). The mathematical model formulation contains the original MCP model as a special case, as can be seen by setting $\theta = 0$ and all $w_i = 1$:

Fig. 17.3 Schematic of the Rosenblatt perceptron with three adjustable weights w_1, w_2 and w_3 and an adjustable bias b . θ is the threshold parameter of the activation function



$$a(\mathbf{x}) = [0, 1, \dots, 1] \cdot \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} = \sum_{i=1}^n x_i. \quad (17.11)$$

This is the formulation for the transfer function of the MCP model; the activation function is the same for both models.

Python Code for the Perceptron Model

The above can be easily implemented in Python, if we assume that the inputs and weights are all numpy arrays. For this, the code of the perceptron model is shown in Python Listing 17.1. The function `predict` makes use of the fact that the

```

1 import numpy as np
2
3 def predict(x, weights, bias, threshold):
4     # Evaluate the transfer function: we're using numpy's scalar product `dot`
5     a = bias + np.dot(x, weights)
6
7     # Activation function: check if the perceptron fires
8     # (For 1 data record a is a scalar and we could simply use
9     # `y = int(a >= threshold)`. For multiple data records, a is a 1D array
10    # --> use numpy's `where(condition, x, y)` function which returns x if
11    # the condition holds and otherwise it returns y.)
12    y = np.where(a >= threshold, 1, -1)
13
14    return y

```

Python Listing 17.1 Implementation of the Rosenblatt perceptron for making a prediction. Here, the prediction will be a value of either 0 or 1. However, it is also possible to use, e.g., -1 and $+1$ (see Sect. 17.2.5 for an explanation)

aggregation function can be computed as a scalar product between $\mathbf{x}$ and $\mathbf{w}$. In the code this is done using Numpy's function `dot` which takes two vectors (or rather: 1D Numpy `array`'s) as arguments. The example usage is for two inputs; for more inputs only the number of elements in the weights and input vectors need to be adapted.

Example 17.2 (Prediction with the perceptron model for two inputs) ✖

Assume a perceptron with two inputs and a threshold of $\theta = 1$. Furthermore, given are the weights $w_1 = 1.5$ and $w_2 = 2$ together with the bias $b = 0.5$. We can then make predictions, e.g., for the input vector $\mathbf{x} = [1, 1]$:

(continued)

```
In [1]: weights = np.array([1.5, 2])
        bias = 0.5
        x = np.array([1, 1])
        y = predict(x, weights, bias, threshold=1.0)
        print(f'x = {x}, y = {y}')
```

```
Out [x]: = [1 1], y = 1
```

`x` contains a single data record with two features. For several data records, `x` has to be a two-dimensional array with features in columns and records in rows.

17.2.2 Again: Boolean Decision Problems

As the MCP model is contained as a special case of the Rosenblatt perceptron, this enhanced model is still able to represent the OR and AND operations. We will now take a look, if the NOT and XOR operators can be represented:

Logical NOT Operation

Starting with the definition of the NOT operator

$$x = 0 \Rightarrow y = \neg x = 1 \quad (17.12)$$

$$x = 1 \Rightarrow y = \neg x = 0 \quad (17.13)$$

and the transfer function for a perceptron with a single input x

$$a(x) = b + wx \quad (17.14)$$

we are looking for the θ that fulfills the inequality

$$a(x) = b + wx \geq \theta \quad (17.15)$$

For $x = 0$ this inequality should be True, i.e., it is $b \geq \theta$. As we have two equations (or rather inequalities) and three unknown variables, θ , b and w , we need to fix at least one of them, and we choose the threshold as $\theta = 0$. For $x = 1$ the inequality Eq. 17.15 should *not* be fulfilled and it reads

$$b + w \not\geq 0 \quad (17.16)$$

$$\Leftrightarrow b + w < 0 \quad (17.17)$$

$$\Leftrightarrow w < -b. \quad (17.18)$$

Table 17.3 Representation of the logical NOT by a perceptron model with one input. The parameters are chosen as $\theta = 0$, $b = 0$ and $w = -1$. The two output columns show the perceptron output and true output

Input		Data Processing		Output
x_1	$a(x) = b + wx$	$a \geq \theta$?	$y = \varphi(a)$	$\neg x$
0	0	$0 \geq 0$?	1	1
1	-1	$-1 \geq 0$?	0	0

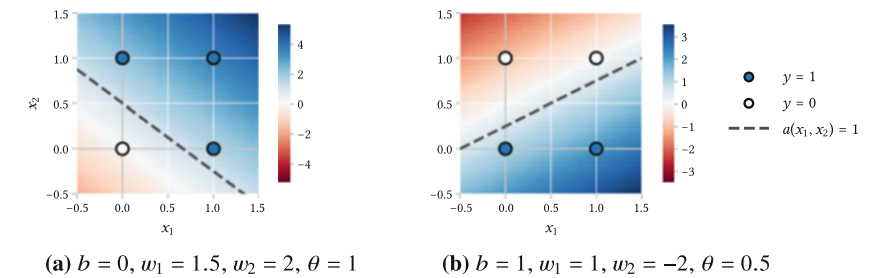


Fig. 17.4 Visualization of response of the perceptron for two different parameter sets. The blue and red color shades show the value of the transfer function $a = b + x_1 w_1 + x_2 w_2$, the dashed line is the “decision boundary” that separates points with values below the threshold θ from those that have values above the threshold. The markers show four particular points; blue filled markers denote those input values x_1, x_2 for which the perceptron would be activated ($y = 1$). (a) $b = 0, w_1 = 1.5, w_2 = 2, \theta = 1$. (b) $b = 1, w_1 = 1, w_2 = -2, \theta = 0.5$

Choosing $b = 0$, it follows that the weight w should be negative, e.g., $w = -1$. With these parameters we get the response as summarized in Table 17.3.

Thus, the perceptron is able to represent the Boolean operation NOT—and therefore potentially also to *learn* such a response (even though so far, it is not clear how “learning” is done for such models).

Logical XOR Operation

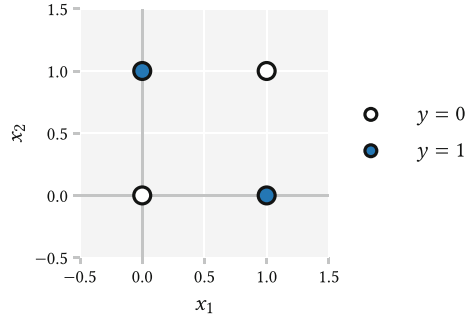
The mathematical proof that the XOR operation is beyond the capability of the perceptron is left as an exercise. Instead of a formal proof we will use a geometrical interpretation in Sect. 17.2.3 to explain why the XOR operation cannot be represented.

17.2.3 Geometric Interpretation

The mathematical model of the perceptron for two inputs can be visualized and interpreted based on geometrical relation. Figure 17.4 shows as a two-dimensional contour plot the values of the transfer function a as a function of x_1 and x_2 for two particular parameter sets b, w_1, w_2 , and θ .

If the inequality of the activation function is further restricted to an equality, i.e.,

Fig. 17.5 If the perceptron would be able to represent the logical XOR operation, then only the points shown as dark markers would have output values of $y = 1$ as shown. However, there exists no straight line that could completely separate the dark from the light markers



$$\varphi(a) = \begin{cases} 1 & \text{IF } a = \theta \\ 0 & \text{ELSE,} \end{cases} \quad (17.19)$$

and we only consider the case $a = \theta$ then the set of equations can be simplified to

$$w_1 x_1 + w_2 x_2 + b = \theta \quad (17.20)$$

$$\Rightarrow x_2 = -\frac{w_1}{w_2} x_1 + \frac{\theta - b}{w_2}. \quad (17.21)$$

This is the equation for a line where the slope is $-w_1/w_2$ and the offset or intercept is $(\theta - b)/w_2$. It is called *decision boundary* as it separates the regions where the neuron fires ($\varphi = 1$) from those where it doesn't fire ($\varphi = 0$) (see Chap. 14 where the notion of the decision boundary was introduced in detail). In analogy, with three (or more) inputs, the separating line turns into a separating plane (or *decision hyperplane*). The decision boundary is shown as dashed line in Fig. 17.4. The visualization of $a(x_1, x_2)$ also reveals that a is a *linear* equation. This has implications for the type of problems that can be represented by the perceptron model: only *linearly* separable data can be learned and properly represented.

What does that imply for the XOR problem? To understand this, we take a look at the left plot of Fig. 17.4. There, only the marker at $x_1 = x_2 = 0$ has a value of $y = 0$, all other points indicate a value of $y = 1$. In terms of logical operations this would represent the OR operation. From this we also can directly see why there is some freedom in choosing the parameters as there exist a multitude of different lines, all of which separate the three blue markers from the white marker. The XOR operator has $y = 1$ if and only if there is exactly one of the input value that has the value 1, i.e., only $x_1 = 1$ and $x_2 = 0$ and $x_1 = 0$ and $x_2 = 1$ result in the output $y = 1$. Figure 17.5 visualizes this configuration.

If the perceptron would be able to represent (and eventually “learn”) the XOR Boolean decision, then there must exist a straight line that separates the points with $y = 1$ from those with $y = 0$. As the figure shows, this is not possible.

Even though, the perceptron is more powerful than the MCP model, it is still not able to make an “either-or” decision, and it is limited to classification problems of linearly separable data.

Historical Comments



In the original publication by ROSENBLATT [8] from 1958 most of the used notions were named in analogy or motivated by the (human) eye and our ability to detect visual patterns:

- What we today denote as *input nodes* was originally named *sensory units* (S-points), e.g., that of a retina which ...
- ... can be *excitatory* or *inhibitory* depending on if they increase or reduce the resulting signal—in our model this is represented by the sign of the respective weight.
- The signals are transmitted to the *association cells* (A-units)—our combined transfer/activation unit.
- The *response units* (R-points) are the output node(s), activated once the threshold is overcome.

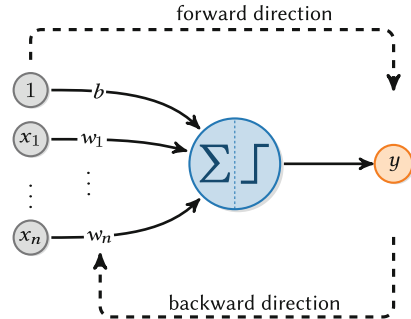
Already ROSENBLATT [7,8] used as variable for the threshold the Greek letter θ , which is why this is still the most common choice.

17.2.4 Supervised Learning with the “Perceptron Learning Rule”

As opposed to the MCP neuron, the concept of the Rosenblatt perceptron includes a feedback loop, where through the so-called *perceptron learning rule* the error is *back propagated*, allowing the weights and bias to be automatically adjusted. The “regular” evaluation of the perceptron, i.e., computing $y^{\text{pred}} = \varphi(\mathbf{x})$, is the *feed forward* step. Figure 17.6 shows the forward calculation direction and the backward direction where the error is “propagated” from the right to the left.

Back-propagation is a crucial component of any *deep learning* (DL) network and will then include the calculation of gradients, but we will come to that later. These are two notions also used in the deep learning networks introduced in one of the following chapters.

Fig. 17.6 The feed-forward consists of evaluating the transfer function and the threshold for given weights, bias, and input values. During back propagation, the weights are automatically adjusted based on the output of the perceptron



Binary Classification



The Rosenblatt perceptron and the MCP model are able to solve classification problems: based on some input data, they can make a decision, and their output data are of categorical type. It is a *binary classification* as there are only two possible outcomes (0 or 1), which, however, can be interpreted, e.g., as “the sample shows failure” or “the sample is intact.”

The perceptron learning rule is a simple approach through which by comparing the desired output with the current output the weights are iteratively changed by small amounts and in proportion to that difference. How strong the influence of the difference is on the updated values is controlled by a scalar factor, the *learning rate*, which we denote in the following by the Greek letter η . If the learning rate η is 0, the weights and bias should not be changed; equally, if the current output value is already the desired output value, the weights and bias should not be changed. Even though it will turn out that this learning rule is unsuitable for a robust learning strategy, the Rosenblatt perceptron is the first attempt of *supervised learning* in the context of early neuronal networks.

Perceptron Learning Rule

Assume that a number of training records in the form of the input features $\mathbf{x}^{\text{train}} = \{x_1^{\text{train}}, \dots, x_n^{\text{train}}\}$ and the response y^{train} are given, and the perceptron should be trained to be able to make predictions. Then, for the training the algorithmic steps shown in Algorithm 17.1 need to be performed.

Algorithm 17.1: Perceptron Learning Rule (for 1 Training Epoch)



1. Initialization:

- a. initialize the weights $w_1, \dots, w_n$ and bias b with zero
- b. choose a suitable learning rate $\eta \in (0, 1]$
- c. prepare a set of data records (e.g., rows from the data matrix)

2. **Feed-forward step:** Choose a new data record and make a prediction for $\mathbf{x}^{\text{train}}$ with the current values of the weights and bias:
 - a. compute the weighted sum $a(\mathbf{x}^{\text{train}}; \mathbf{w}, b)$
 - b. evaluate the activation function: $y^{\text{pred}} = \varphi(a; \theta)$

3. **Backward propagation:** use difference between predicted and true output to compute the update increments for weights and bias

$$\Delta \mathbf{w} = (y^{\text{train}} - y^{\text{pred}}) \mathbf{x}^{\text{train}} \quad \text{and} \quad \Delta b = y^{\text{train}} - y^{\text{pred}} \quad (17.22)$$

Update the weights and bias to obtain the new values

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \Delta \mathbf{w} \quad \text{and} \quad b \leftarrow b + \eta \Delta b \quad (17.23)$$

4. **GO TO step 2 and repeat** with a new record $(\mathbf{x}^{\text{train}}, y^{\text{train}})$ until all data records were used exactly once.

Epoch

Any full iteration, consisting of both forward steps and weight/bias updates for *all* data records, is called an *epoch*. A complete training process usually consists of several epochs where each new epoch starts with the updated weights and bias values from the previous epoch. Typically, for a very small learning rate, more epochs are required than for larger learning rates. Often, it is the goal to use a learning rate that is as large as possible so that only few epochs are required for training, which makes the training process fast. At the same, if the learning rate is too high, numerical instabilities will occur and the training process will not be able to converge.

Termination Criterion

The training is terminated either if an error is sufficiently small (see “error measures” below) or if a maximum number of training epochs is reached. The latter is helpful for avoiding that the algorithms encounter a situation where the error cannot be reduced sufficiently and where one would end up in an “infinite loop.” An obvious choice is termination if the error is zero. However, it can be formally proven that the perceptron learning algorithm only converges if the dataset consists of two linearly separable classes. Therefore, a more generous minimal error is often more useful and should be chosen based on the dataset and the application.

The Peculiarity of the Perceptron Learning Rule



The perceptron learning rule is somewhat unusual as it does an individual forward/update step for *each* of the training data records. Therefore, in particular for small datasets, the order of the data records matters. This is an important difference to, e.g., the ADALINE algorithm introduced in Sect. 17.3: there, the weights and bias update is determined based on all data records at once.

Error Measures

How can the error be measured for the perceptron learning rule? Assuming that the number of data records is M , the *absolute error* (AE) for the classification can be easily computed as the total number of misclassified data,

$$\text{absolute classification error AE} = \sum_{i=1}^M \left| y^{\text{true}}_i - y^{\text{pred}}_i \right|, \quad (17.24)$$

where y^{true}_i is the true class label of data record number i and y^{pred}_i is corresponding predicted class label. This error measure can only take integer values that represent the number of misclassified data and can therefore be easily interpreted. Another useful error measure, that we already encountered in Sect. 11.7, is the mean absolute error (MAE), i.e., the number of mislabeled data divided by the number of samples

$$\text{mean abs. classification error MAE} = \frac{1}{M} \sum_{i=1}^M \left| y^{\text{true}}_i - y^{\text{pred}}_i \right|. \quad (17.25)$$

This allows to specify an error tolerance that is independent of the size of the dataset, which can be used to improve the termination criterion of the algorithm. For example, one could weaken the originally very strict requirement that all samples need to be correctly predicted and postulate that not more than 5% of the data should be misclassified.

A comment on “Learning”



As we saw in the previous chapters, *learning* typically requires a number of *examples* from which the *machine learning* (ML) model is supposed to generalize to new data. This has already been formulated by ROSENBLATT [8], who in his early publications was wondering about the amount of

(continued)

information needed to recognize a geometrical shape: the strength of a good and predictive model is *not* to learn *all* possible sizes and shapes of a triangle but rather to learn the most important characteristics from a finite number of examples.

Python Implementation of the Perceptron

A numerical implementation that uses the absolute classification error as a termination criterion can be found in Python Listing 17.2. The function `train_1_epoch`

```

1 import numpy as np
2 from perceptron_prediction import predict
3
4
5 def train_1_epoch(X_train, y_train, weights, bias, threshold, learning_rate):
6     n_samples, n_features = X_train.shape
7     y_pred = np.zeros_like(y_train)
8
9     for j in range(n_samples):
10         # feed forward direction
11         y_pred[j] = predict(X_train[j], weights, bias, threshold)
12
13         # backwards direction ("perceptron rule")
14         weights += learning_rate * (y_train[j] - y_pred[j]) * X_train[j]
15         bias += learning_rate * (y_train[j] - y_pred[j])
16
17     return weights, bias
18
19
20 def train(X_train, y_train, threshold, n_epochs, learning_rate):
21     n_features = X_train.shape[1] # the features are the columns of X_train
22     weights = np.zeros(n_features)
23     bias = 0.
24
25     for i in range(n_epochs):
26         weights, bias = \
27             train_1_epoch(X_train, y_train, weights, bias,
28                           threshold, learning_rate)
29
30         y_pred = predict(X_train, weights, bias, threshold)
31         if np.sum(np.abs(y_pred - y_train)) == 0:
32             break
33
34     return weights, bias
35
36
37 # The following two lines show how to use the code:
38 X_train, y_train = create_training_data() # X_train is a 2D array, y_train is 1D
39 weights, bias = train(X_train, y_train,
40                       threshold=0.4, n_epochs=50, learning_rate=0.5)

```

Python Listing 17.2 Implementation of the perceptron learning rule. The code reuses the function `predict` from Python Listing 17.1. Here, the training is stopped if the error is zero. Note that contrary to all best practices in programming, we omitted docstrings, comments, type hints, and “sanity checks” to ensure that the code stays compact

performs a complete training step consisting of the prediction and the back-propagation for all samples of the dataset and returns the updated weights and bias. The function `train` does the training for `n_epochs`. `x_train` is a 2D array and `y_train` is a 1D array; `x_train.shape` returns the number of rows (number of samples) and columns (number of features) of the 2D array. For each epoch, the updated weights and bias values are computed and, based on that, the `AE` calculated. For this, numpy's `abs` functions compute the absolute value *element-wise* for each entry of the vector `y_pred - y_train`. These values are then summed up. If the error is zero, then the training is stopped and the `break` statement quits the `for` loop. Note that changing the threshold can be counteracted by changing the bias. This is why by fixing the bias to zero, one can say, that the algorithm is also learning the threshold.

To check if the code works properly, a first “sanity check” is to predict the training data: for `x_train` we already know that the prediction should be `y_train`.

Convergence Theorem

Taking a look at Eq. 17.22, one observes that once a solution $\mathbf{w}$ has been found that correctly predicts all training data, then $\mathbf{w}$ will never be changed again. A formal proof is given by [9] and shortly afterward by [4] including a formal discussion of the limitations of the Rosenblatt perceptron.

17.2.5 Geometric Interpretation of the Perceptron Training

Interpretation of the Weights Vector

Point of departure for this part is the mathematical representation of lines. Already above we saw that the decision boundary for the case of a zero threshold is given by

$$\mathbf{w} \cdot \mathbf{x} + b = 0, \quad (17.26)$$

which is the equation of a line (or hyperplane if $\mathbf{x}$ is a position vector in an arbitrary dimensional space). A non-zero threshold value can be included in an adjusted value of b . From this form the normal of the line can be easily inferred: two vectors $\mathbf{a}$ and $\mathbf{c}$ are normal if and only if the scalar product $\mathbf{a} \cdot \mathbf{c} = 0$ (cf. Appendix A). To find a normal vector for a given vector $\mathbf{x}$ we therefore have to find a vector $\mathbf{a}$ such that $\mathbf{a} \cdot \mathbf{x} = 0$. Neglecting b in Eq. 17.26 we see that such a vector would be $\mathbf{w}$. This is still true if we add b , as this only acts as an offset of the line but does not change the direction. Therefore, $\mathbf{w}$ is a vector that is perpendicular to the line given by Eq. 17.26.

Choice of y Values

Sometimes the two classes are not chosen to be represented by the values 0 and 1 but rather by -1 and 1 . Both variants work equally well and result in identical training behavior if the learning rate is scaled accordingly. The former convention is rather motivated by Boolean decision problems where 0 and 1 have a clear correspondence

to the Boolean values of `False` and `True`. Additionally, for problems with more classes it is easy to simply count them starting from 0. The second convention is motivated by the fact that for the geometrical interpretation, the normal to the decision boundary, $\mathbf{b}$, points into the area, where correctly classified y values are *positive*. Those values in the area pointed at by the vector $-\mathbf{b}$ would be negative. For the rest of this section, we follow the second convention.

Geometry of the Perceptron Update

How is this important for understanding how the perceptron learning rule works? Figure 17.7a shows a hypothetical configuration with a given training dataset $\mathcal{D}$. According to Algorithm 17.1, a point number i (or equivalently the data record i) is chosen, and the corresponding position vector is $\mathbf{x}^{(i)}$. The current weights and bias result in the shown decision boundary and the normal vector $\mathbf{w}$. The update from the above algorithm reads for data record number i

$$\mathbf{w}^{\text{new}} := \mathbf{w}^{\text{old}} + \eta \left(y^{(i)} - \hat{y}^{(i)} \right) \mathbf{x}^{(i)}, \quad (17.27)$$

where we used the short form $y^{(i)} := y^{\text{true}}$ and $\hat{y}^{(i)} := y^{\text{pred}}$ to avoid too lengthy superscripts.

The point i is misclassified with $\hat{y}^{(i)} = +1$ and the term in parenthesis evaluates to $1 - (-1) = 2 > 0$. With $\lambda = 2\eta$ the new weight vector is then $\mathbf{w}^{\text{old}} + \lambda \mathbf{x}^{(i)}$ —addition of two vectors as shown in Fig. 17.7b. At the same time, the value of the bias is incremented by λ . This results in a rotation of the decision boundary in clockwise direction, cf. Fig. 17.7c. If the misclassification of a triangular marker would have been considered, then the factor λ would be negative, which results in a rotation of the decision line in the opposite, counter clockwise direction. If a point was correctly classified, then the decision boundary does not change. A perceptron model without a bias would have a hyper-plane that always goes through the origin.

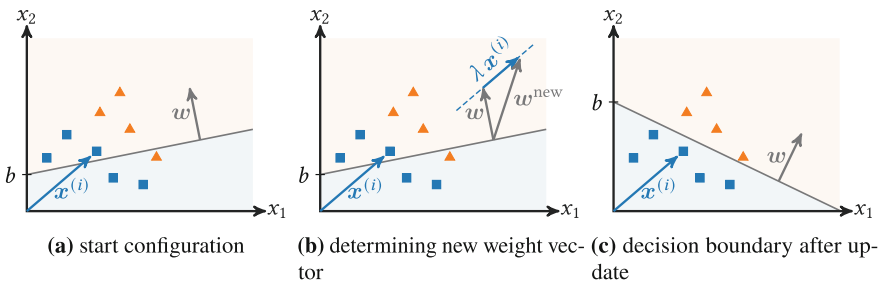


Fig. 17.7 Illustration of the geometrical relations implied by the perceptron learning rule. The decision boundary is given by $\mathbf{w}$ and b , cf. (a). There, three points are misclassified: two rectangular and a triangular point. An arbitrary point $\mathbf{x}^{(i)}$ is chosen that is then used for the update step. The new weight vector in (b) is the sum of the current $\mathbf{w}$ and a vector along the direction of $\mathbf{x}^{(i)}$: its length is proportional to the training rate as indicated by the factor λ . Additionally, the sign of the line direction is determined by the difference $y^{\text{true}} - y^{\text{pred}}$

17.3 The ADALINE Model

The *Adaptive Linear Neuron* (ADALINE) (also sometimes called “Adaptive Pattern Classification Machine”) was developed in the 1960s by BERNARD WIDROW and TED HOFF [12]. It is also based on the idea of the MCP neuron and the Rosenblatt perceptron. However, the novelty of ADALINE is an enhanced training strategy as compared to the perceptron. Below, we will see that the training strategy is in fact an efficient gradient descent method for finding optimal weights.

17.3.1 ADALINE Training Strategy

The key difference between the Rosenblatt perceptron and the ADALINE model is the way how the weights are adjusted: while the Rosenblatt perceptron used the output y to determine the training increments for bias and weights, the ADALINE model directly uses the agglomerated and weighted input values $\mathbf{w} \cdot \mathbf{x} + b$. This can be best seen in the schematic shown in Fig. 17.8.

There, after summation we have now an activation function, which here is just taken as the identity function $\varphi(x) = x$. The output is squeezed through a “quantizer”³—nothing but a threshold function, which, however, gives as output either -1 or $+1$ (see below for the motivation). The backward direction computes the error based on the output of the activation function according to the so-called *Widrow & Hoff learning rule*, which consists of the following increments

$$\Delta \mathbf{w} = (y^{\text{train}} - a(\mathbf{x}^{\text{train}}; \mathbf{w}, b)) \mathbf{x}^{\text{train}} \quad \text{and} \quad \Delta b = (y^{\text{train}} - a(\mathbf{x}^{\text{train}}; \mathbf{w}, b)). \quad (17.28)$$

$a(\mathbf{x}^{\text{train}}; \mathbf{w}, b)$ is the output of the (identity) activation functions for given weights and bias. Update of the weights and bias is then done using the learning rate η by the update step

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \Delta \mathbf{w} \quad \text{and} \quad b \leftarrow b + \eta \Delta b \quad (17.29)$$

where these equations correspond to the perceptron’s Eqs. 17.22 and 17.23. Additionally, the update is not done incrementally for each record or sample of the dataset. Instead, the two increments are computed for the whole dataset such that the update is done for all records simultaneously. Everything else stays the same as for the perceptron learning rule. A summary of the whole algorithm can be seen in Algorithm 17.2.

³ In signal processing, “quantization” denotes the process of mapping continuous data to discrete data.

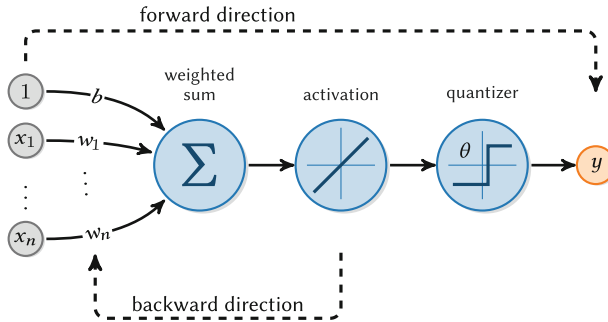


Fig. 17.8 Schematic of the ADALINE model: The forward direction implies evaluating the transfer, activation, and “quantizer” function for given weights, bias, and input values. For the backward direction, the error from the activation function (here: the identity function) is used to automatically adjust weights and bias

Algorithm 17.2: ADALINE Training with the Widrow & Hoff Learning Rule



1. **Initialization:** (training data must have $y^{\text{train}} \in \{-1; 1\}$)
 - a. initialize the weights $w_1, \dots, w_n$ and bias b with zero
 - b. choose a suitable learning rate $\eta \in (0, 1]$
 - c. define a maximum number of epochs (=iterations) as termination criterion; define an error tolerance as convergence criterion
2. **Forward step:** Make *simultaneous* predictions for *each* record of the dataset with current weights and bias:
 - a. compute the weighted sum $a(\mathbf{x}^{\text{train}}; \mathbf{w}, b)$
 - b. evaluate the activation function (here: identity function): this gives again the weighted sum a
 - c. evaluate the quantizer function: $y^{\text{pred}} = \varphi(a)$
3. **Backward direction:** use difference between true output, y^{train} , and the output values of the activation function, a , to compute update increments for weights and bias

$$\Delta y = y^{\text{train}} - a(\mathbf{x}^{\text{train}}; \mathbf{w}, b) \quad (17.30)$$

$$\Delta \mathbf{w} = \Delta y \mathbf{x}^{\text{train}} \quad \text{and} \quad \Delta b = \Delta y \quad (17.31)$$

Update the weights and bias to obtain the new values

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \Delta \mathbf{w} \quad \text{and} \quad b \leftarrow b + \eta \Delta b \quad (17.32)$$

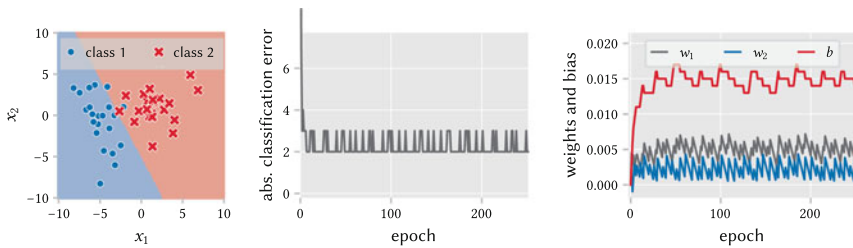
4. **GO TO step 2 and repeat** until the norm of the error, $\|\Delta y\|$, is small enough or until the maximum number of iterations is reached.

17.3.2 Example: Comparison Between the Perceptron and the ADALINE Model

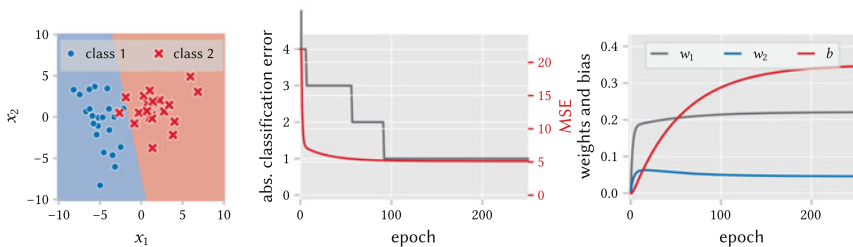
We now take a look at how well the perceptron and the [ADALINE](#) perform for a dataset that consists of two data classes, which have a small amount of overlap and are therefore, strictly speaking, not linearly separable. This was motivated by the fact that often in many real problems, some of the assumptions used for the models are slightly violated, but obviously a good model has a certain degree of tolerance.

In Fig. 17.9, we show the training results for the perceptron and the [ADALINE](#) model. All parameters, i.e., the training data itself, learning rate, initial values for bias and weights, are chosen identically. There, it becomes obvious that the [ADALINE](#) is numerically much more stable, which is the main side effect of using the output of the (continuous) activation function and not the output of the step functions.

For the perceptron, the error keeps fluctuating and does not reach the same small error as the [ADALINE](#). The decision boundary keeps jumping and rotating, which is also not fundamentally changing if the learning rate is further decreased. Also, the [ADALINE](#) model is not able to perfectly classify all the data, but this is not surprising



(a) Training of the Perceptron



(b) Training of the ADALINE model (additionally, the MSE error is shown in the middle panel)

Fig. 17.9 Comparison of the training results for the Perceptron and the [ADALINE](#) model. Top panel: final classification state, middle: error, bottom panel: evolution of weights and bias. The perceptron oscillates and only reaches a larger error than the [ADALINE](#) model. The [ADALINE](#) model is also numerically more stable. (a) Training of the Perceptron. (b) Training of the ADALINE model (additionally, the MSE error is shown in the middle panel)

as it is a *linear* model. Overall, it looks numerically much more well-behaved and converges to a set of weights and bias that result in the minimum possible error.

Is it surprising or problematic that the weights and bias values of the two models have different magnitudes? For almost all problems, there is no unique solution to the problem of finding a decision boundary. Thus, there exists a potentially large range of parameters that work equally well. Additionally, a mathematical line is fully determined by two parameters, whereas the two studied models contain two weight parameters and the bias and therefore have one degree of freedom more than needed.

17.3.3 The Relation to Gradient Descent Optimization

The [ADALINE](#) training uses the linear activation function a for computing an error. This has the advantage that a is differential, and that we can define a cost function $J(\mathbf{w})$ that can then be minimized. Here and in the remainder of this paragraph, we assume that the vector $\tilde{\mathbf{w}}$ contains both the weights and the bias, $\tilde{\mathbf{w}} := [w_1, \dots, w_n, b]^T$ as introduced in Eq. 17.8, which also required a new definition of the input vector as $\tilde{\mathbf{x}} := [1, x_1, \dots, x_n]^T$. Then we define the cost function as the sum of the squared errors of all M data records,

$$J(\tilde{\mathbf{w}}) = \frac{1}{2} \sum_{i=1}^M (y_i - a(\tilde{\mathbf{x}}_i; \tilde{\mathbf{w}}))^2. \quad (17.33)$$

$\mathbf{x}_i$ is the i -th data record, and the factor of one half is here to simplify the formulation after taking the derivative below and does not have any further influence. Minimizing the cost function w.r.t the weight components then yields the values of the trained weights. In the following, we use the simple gradient descent method (cf. Chap. 12 for further details). There, we should follow the direction of the steepest descent, which is $-\nabla J(\mathbf{w})$. Rewriting this component-wise lets us explicitly perform the differentiation along the j -th component:

$$\frac{\partial J}{\partial \tilde{w}_j} = \frac{1}{2} \frac{\partial}{\partial \tilde{w}_j} \left(\sum_{i=1}^M (y_i - a(\tilde{\mathbf{x}}_i; \tilde{\mathbf{w}}))^2 \right) \quad (17.34)$$

For taking the partial derivative of the lengthy expression, we note that the derivative of a sum equals the sum of the derivatives. Therefore, we can move the partial differentiation into the summation and apply it to the $(\dots)^2$ term using the chain rule. This results in

$$\frac{\partial J}{\partial \tilde{w}_j} = \sum_{i=1}^M (y_i - a(\tilde{\mathbf{x}}_i; \tilde{\mathbf{w}})) \left(-\frac{\partial a(\tilde{\mathbf{x}}_i; \tilde{\mathbf{w}})}{\partial \tilde{w}_j} \right). \quad (17.35)$$

For taking the partial derivative of a with respect to any $\tilde{w}_j$, recall that with the extended notation of $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{w}}$ it is $a_i = \sum_j \tilde{w}_j \tilde{x}_{ij}$ (hint: recall the definition of the data matrix, Sect. 3.3.3). Therefore, there is only one element of the sum that depends on $\tilde{w}_j$, which would be the j -th element. Then, it is $\partial a_i / \partial \tilde{w}_j = \tilde{x}_{ij}$ with which follows that

$$\frac{\partial J}{\partial \tilde{w}_j} = - \sum_{i=1}^M (y_i - a(\mathbf{x}_i; \tilde{\mathbf{w}})) \tilde{x}_{ij} = - \sum_{i=1}^M (y_i - \tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}}) \tilde{x}_{ij} \quad (17.36)$$

This equation gives the sensitivity of the cost function with respect to the j -th weight. Writing this in compact vectorial form for *all* weights eliminates the occurrence of the index j and reads:

$$\frac{\partial J}{\partial \tilde{\mathbf{w}}} = - \sum_{i=1}^M (y_i - \tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}}) \tilde{\mathbf{x}}_i. \quad (17.37)$$

This is, by construction, a vector that points into the direction of the steepest descent. For the iterative update of the search direction, the gradient descent method then takes a small step along this new search direction, resulting in the weights increment

$$\Delta \tilde{\mathbf{w}} = -\eta \nabla J(\tilde{\mathbf{w}}) = \eta \sum_{i=1}^M (y_i - \tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}}) \tilde{\mathbf{x}}_i \quad (17.38)$$

$$\Leftrightarrow \Delta \tilde{w}_j = -\eta \frac{\partial J}{\partial \tilde{w}_j} = \eta \sum_{i=1}^M (y_i - \tilde{\mathbf{x}}_i \cdot \tilde{\mathbf{w}}) \tilde{x}_{ij}. \quad (17.39)$$

Note that for comparison with the delta rule of the [ADALINE](#) model, we have to convert the extended notation $\tilde{\mathbf{w}} = [w_1, \dots, w_n, b]^\top$ back to $\mathbf{w}$ and θ . This is the same update formula that is used from the [ADALINE](#) model above which also shows why [ADALINE](#) is indeed based on the gradient descent optimization method.

We conclude this derivation by mentioning that $\partial J / \partial \mathbf{w}$ is not necessarily the direction of the minimum. However, for problems that exhibit only a global minimum and no local minima, the gradient descent method can be shown to always converge to the global minimum. For cost functions that have local minima, this guarantee is only restricted to the vicinity of the global minimum.

17.3.4 Limitations of the Perceptron and the ADALINE Model

We have seen that both models—even the simplistic perceptron—work very well for linearly separable data. This gave rise to great hope that with such models truly artificial intelligent machines could be created. This hope was vastly destroyed in 1969 by Minsky and Papert [4], who in their book formally proved that no linear,

single layer model is able to learn the simple logical XOR operator. This limitation (among others) was so severe, that after the rapid progress and enthusiasm of the 1950s and 1960s, further developments were dramatically reduced until the 1980s, a period of which is sometimes called *AI winter*.

17.4 Increasing the Complexity: Assemblies of Neurons

From an early point in time on, it was clear that the perceptron and the [ADALINE](#) model were very limited in terms of learnable the degrees of complexity. In fact, already the very early units were combined to form assemblies of perceptrons to increase the complexity. Again, the motivation came from the biological nervous system. In the following, we first analyze networks of neurons and discuss what kind of functions can be represented by them. In a next step, we then turn to the investigation of aspects related to the training of such assemblies.

17.4.1 A First Network of Two Computational Neurons

The obvious approach is to combine multiple perceptrons in order to increase the complexity of the model and—hopefully—thereby also to increase the complexity of the predictable phenomena. In Fig. 17.10a, two neurons with n input values/features are combined.

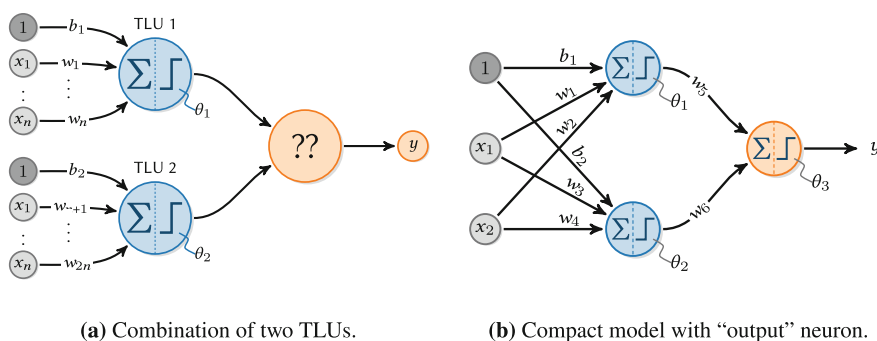


Fig. 17.10 Combination of two perceptrons: (a) The output of two perceptrons with n input nodes will be combined in the “??”-node. This gives then the output y , a real value or a categorical value (class). (b) is a more compact notation with input nodes that are “reused” for each neuron; their output is summed and tested against a threshold value (the large orange node), yielding the output y that is used for classification. For a multi-class classification problem, more output nodes would be used; for regression problems, there would be no thresholding component in the output node

Example with 2 Perceptrons

The output of each of the two perceptrons is fed as input into an output unit that—depending on whether a regression problem, a binary or multi-class classification problem is to be solved—has different functionality.

We turn now to a concrete example: a network of two perceptrons, each of which has two inputs, to be used for a classification problem. This is shown in Fig. 17.10b where we also changed to a commonly used depiction: the two identical input nodes of the perceptrons are merged, and from every input node connections go to both units; their output is not depicted as an individual node.⁴ We assume that each of the three shown units consists of a summation and activation function of the form $a(\mathbf{w} \cdot \mathbf{x}; \theta)$ where θ is again a threshold (the notation with the semicolon indicates that θ is a constant parameter while everything before the semicolon is considered a variable. Sometimes, the parameters are omitted for brevity). We can then start from the right of Fig. 17.10b to recursively derive the formulation for y :

$$y = a_3(y_1 w_5 + y_2 w_6; \theta_3) \quad (17.40)$$

where y_1 and y_2 are the output values from the two TLUs. They can be computed as

$$y_1 = a_1(b_1 + x_1 w_1 + x_2 w_2; \theta_1) \quad (17.41)$$

$$y_2 = a_2(b_2 + x_1 w_3 + x_2 w_4; \theta_2) . \quad (17.42)$$

The a_i are the transfer and activation functions with the threshold values θ_i . The above equations are already sufficient to perform predictions if the weights and the bias values are given. In Example 17.3, a step-by-step calculation is performed for the case of two perceptrons.

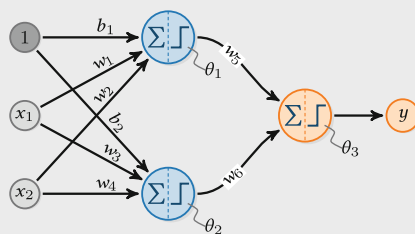
Example 17.3 (Prediction with two Perceptrons)



The following figure depicts a simple network of two combined perceptrons and an output unit:

(continued)

⁴ This is a convention that exists mainly for historical reasons. Additionally, in the context of connected neurons, only one node is needed for storing the value, e.g., in case that the output of the one is the input of the other.



It requires altogether six weights and two bias values, which are given as:

$$\begin{aligned} w_1 &= -2.0, & w_2 &= 2.0, & b_1 &= 0.5 \\ w_3 &= 1.5, & w_4 &= -2.1, & b_2 &= 0 \\ w_5 &= -1.8, & w_6 &= -2.2 \end{aligned}$$

together with the threshold values $\theta_1 = -1$ and $\theta_2 = -1$ for the two perceptrons and $\theta_3 = -2.5$ for the output unit (which we define to have values of either 0 or 1). What is the output y for a given input of $x_1 = 0.8$ and $x_2 = 0.3$?

We start by evaluating Eq. 17.41:

$$y_1 = a_1(0.5 + 0.8 \cdot (-2.0) + 0.3 \cdot 2.0) = a_1(-0.5)$$

To compute the first activation function $a_1(-0.5)$, we compare the function's argument with the threshold $-0.5 \geq \theta_1 = -1$ from which follows that $y_1 = a_1(-0.5) = 1$. In analogy, we find for Eq. 17.42 that $y_2 = a_2(0.57) \geq \theta_2 = -1$. From this follows that $y_2 = 1$. Finally, we compute the overall output from Eq. 17.41 via $a_3(-1.8 \cdot 1 + (-1.8) \cdot 1)$ and the comparison with θ_3 resulting in $y = 0$.

Is this a network with 2 or 3 neurons?



The network in Example 17.3 seems to show altogether three neurons but we are only talking about two—was there a mistake?

Well, even though in the figure, y is computed as the output of a perceptron with the same mathematical form as the two first units, it is composed of a distinct function. This will become more clear, e.g., in case of a multi-class classification or regression problem where this unit will take a different mathematical form. This is why this network is considered a “2-neuron network.” We will also discuss this when we analyze the decision boundary of such a network.

17.4.2 The Boolean XOR Decision Problem

Is this simple network able to perform better on the Boolean XOR problem than a single neuron? Using the same network structure as in Example 17.3, we perform computations with all possible permutations of 0s and 1s as input, as shown in Table 17.4 (note that there we used the parameters given in the caption of the table). Then, it is easy to show that the Boolean XOR operator can indeed be represented by this network.

Note that the output of the two neurons is -1 or $+1$, whereas the final output y will be either 0 or 1 as we wanted to represent the Boolean states `False` and `True` (another hint that the output unit is different from the two neurons).

Why are two neurons able to solve the XOR problem and in which aspects is their combination different from a single neuron? To understand this, we visualize the function a_3 depending on the input x_1 and x_2 as shown in Fig. 17.11 (recall that in Eq. 17.40 the variables y_1 and y_2 depend on x_1 and x_2 and therefore y implicitly depends on them as well).

The “decision-making” of the two individual TLUs shows the expected linear decision boundary separating regions of $y_1(x_1, x_2) = -1$ from those with $y_1(x_1, x_2) = +1$ (y_2 in analogy). These two regions are also called *half-spaces* as they start at the decision boundary and from there continue infinitely. The output node essentially adds weighted versions of y_1 and y_2 and then compares it with a threshold value. In our case, the values of the weights w_5 and w_6 are not too different so that effectively y_1 and y_2 are averaged, as can be clearly seen in the bottom left plot of Fig. 17.11. The values between which the output node should choose are expected to represent Boolean values. Therefore, we set it such that the values can only be 0 or 1.

In conclusion, we observe that the composition of two perceptrons effectively combines two decision boundaries, which allows to create a more complex resulting decision boundary that is not restricted to linearly separable problems and—finally—is able to solve the Boolean XOR decision problem. Do we also have to show that the other Boolean decision problems (AND NOT and OR) can still be

Table 17.4 Representation of the logical XOR by two connected perceptrons. Parameters used are $w_1 = 0.7$, $w_2 = -1.1$, $b_1 = 1$, $w_3 = -w_4 = 2$, $b_2 = 0$, $\theta_1 = 0.4$, $\theta_2 = 0.8$, $w_5 = -w_6 = 0.5$, $b_3 = 0$, $\theta_3 = 0.1$. The two output columns show the model output as well as the true output $x_1 \dot{\vee} x_2$

Input				Data Processing								Output
x_1	x_2	a_1	$a_1 \geq \theta_1 ?$	y_1	a_2	$a_2 \geq \theta_2 ?$	y_2	a_3	$a_3 \geq \theta_3 ?$	$y = \varphi(a_3)$	$x_1 \dot{\vee} x_2$	
0	0	1.0	True	1	0.0	False	-1	1.0	True	0	0	
1	0	1.7	True	1	2.0	True	1	0.0	False	1	1	
0	1	-0.1	False	-1	-2.0	False	-1	0.0	False	1	1	
1	1	0.6	True	1	0.0	False	-1	1.0	True	0	0	

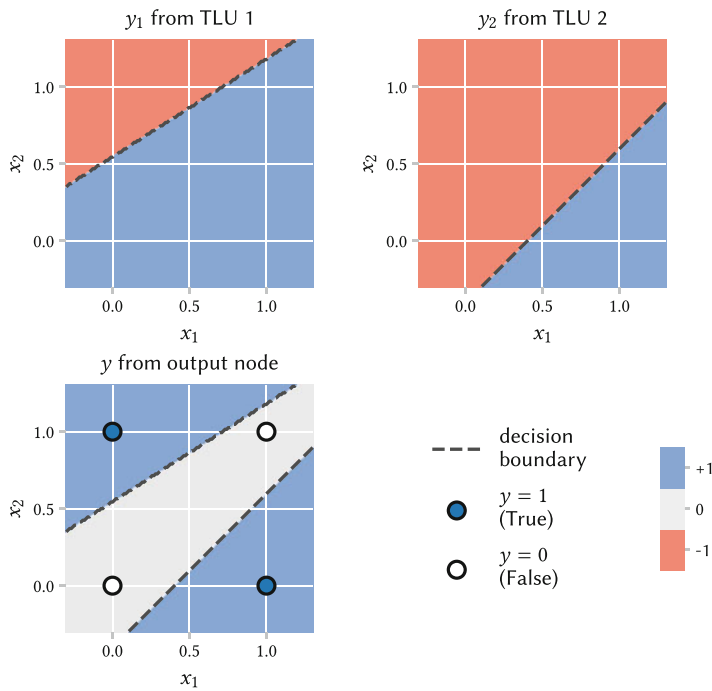


Fig. 17.11 Results from an assembly of two perceptrons with parameters according to Example 17.3. The weights, bias, and threshold values are chosen as in Table 17.4. The top left and top right panels are the intermediate outputs of neuron 1 and 2, the bottom panel shows the results of the output node. y_1 and y_2 are the output of perceptrons and are either -1 or $+1$. As the output y should represent Boolean values, it was here defined to be either 0 (False) or 1 (True). In the bottom left panel, the composition of the effect of the two perceptrons becomes directly visible in the resultant decision boundaries

represented? The proof that this still holds is simple as we can set one of the weights of the output unit to zero such that effectively only one unit is contained.

17.4.3 Python Implementation of the Two-Neuron-Assembly

In Python Listing 17.3, an implementation of the assembly of two neurons is shown, which is intuitively understandable and matches the mathematical formulation. There, we don't even reuse functions from the previous sections.

Why not directly implement this for an arbitrary number of inputs or neurons? Because such code would not be very flexible, e.g., if the number of neurons or inputs changes, a substantial part of it needs to be rewritten. The purpose of this code example is twofold: First of all, it shows the information flow very clearly and thereby paves the way for more complex and general networks introduced in the following sections. The second important aspect is that we can use these lines of

```

1 import numpy as np
2
3 b1, w1, w2, theta1 = 0.5, -2.0, 2.0, -1.0 # TLU 1
4 b2, w3, w4, theta2 = 0.0, 1.5, -2.1, -1.0 # TLU 2
5 b3, w5, w6, theta3 = 0.0, -1.8, -2.2, -2.5 # output node
6
7 # the 4 points for the XOR problem: (0, 0), (1, 0), (0, 1), (1, 1)
8 x1 = np.array([0, 1, 0, 1])
9 x2 = np.array([0, 0, 1, 1])
10
11 # TLU 1
12 a1 = w1 * x1 + w2 * x2 + b1 # weighted sum
13 y1 = np.where(a1 >= theta1, +1, -1) # evaluate activation functions
14
15 # TLU 2
16 a2 = w3 * x1 + w4 * x2 + b2 # weighted sum
17 y2 = np.where(a2 >= theta2, +1, -1) # evaluate activation functions
18
19 # Output node
20 a3 = w5 * y1 + w6 * y2 + b3
21 y = np.where(a3 >= theta3, False, True)
22
23
24 print(f'x1 = {x1}, x2 = {x2}')
25 print(f'y = {y}')
26 # [output]: x1 = [0 1 0 1], x2 = [0 0 1 1]
27 # [output]: y = [ True False False  True]

```

Python Listing 17.3 An implementation of the network of two neurons. Even though it is a very simplistic implementation, it is able to handle a number of input records at the same time as can be seen by the fact that `x1` and `x2` are both numpy `array`s

code to produce “reference solutions” to be used as validation of more complicated networks.

Parameters used are those for solving the XOR problem from above. With the given comments, the code should be mainly self explanatory. We note, that `a1` in line 9 is again an `array` where each entry corresponds to one of the four data record of the input `x1` and `x2`. `y1` uses numpy’s `where` function that gives `+1` for each element that fulfills the condition (e.g. `a1 >= theta1`) and `-1` otherwise. The function `where` is able to do this for all elements of an array.

Words of advice



In case that the reader skipped the sections on vectorization in the chapters on regression (Sect. 12.4) we suggest to try to understand the “vector operations” in Python Listing 17.3 (i.e., the way how the four data records in `x1` and `x2` are handled). This will be needed for many of the subsequent implementations. Printing the output of each line and adding more entries to `x1` and `x2` (=increasing the number of data records) might also be helpful.

17.4.4 A Fully Connected Network with Input, Output and a Single Hidden Layer

Already a few years after ROSENBLATT published the perceptron [8] he also published the concept of a *multilayer perceptron* (MLP), which contained an input layer, a hidden layer, and an output layer [9] (“layers” denote arrays of similar neurons or nodes, stacked on top of each other, as shown in Fig. 17.12). The hidden layer was initialized with random values for the weights. However, there was no mechanism such that the hidden layer was able to adjust the weights. MLPs has been used in many variations and could also be studied mathematically. They are a predecessor of DL, and therefore, we will introduce them now in somewhat more detail.

Starting from single units, we can assemble more complex structures in the same spirit as when we assembled two perceptrons above. As the assemblies are now getting more complex, also the notations become more and more involved. As this can quickly become overwhelming (who likes dealing with variables with four or five indices?!), we first take an intermediate step considering a network with only *three* different types of “layers.” Figure 17.12 shows an example of such a network.

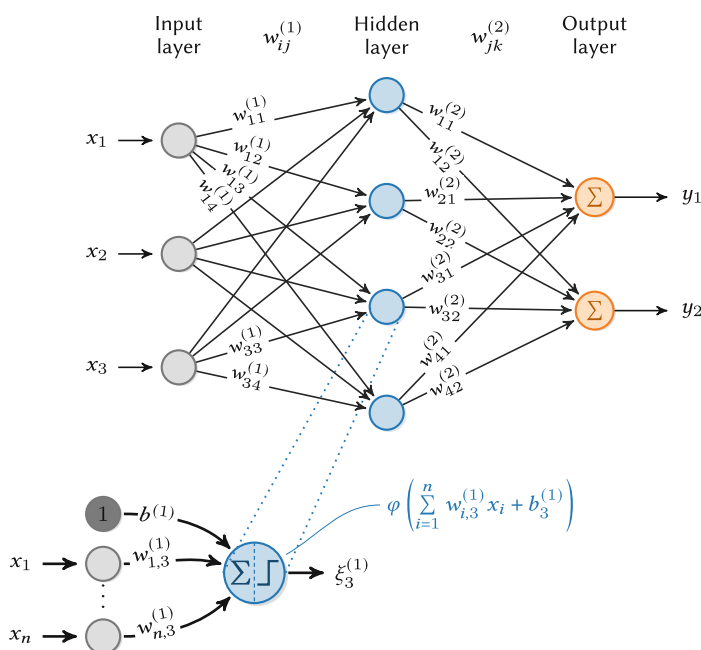


Fig. 17.12 Example of a neural network with one input layer (containing three input nodes), a hidden layer (with four perceptrons), and an output layer (with two output nodes). On the right is the symbolic representation that is used in the following and that is in detail explained in the text. Note that the “bias input node,” usually indicated by the “1,” is not shown but implicitly assumed to be always present. Two distinct sets of weights are present, the $w_{ij}^{(1)}$ and the $w_{jk}^{(2)}$

There are two layers that are visible or accessible from the outside of the model: the input layer containing nodes that store the input data x_i , and the output layer collecting data and returning them in form of the values y_j . In between is a hidden layer that is not visible from outside the model. All layers can have an arbitrary number of nodes or neurons. Note that there is a fundamental difference between the nodes of the input and the output layer: the output nodes perform computations, while the nodes of the input layer only store data.

Words of advice



Naming conventions are often not standardized and rather based on “common agreements” or habits. For example, a number of different names exist for the network shown in Fig. 17.12: sometimes it is called a *two-layer* network as there are *two* layers that do actual computations the hidden and the output layer (or because there are two “layers of weights”), but it really *consists* of three layers of nodes (the input, hidden, and output layer). Sometimes the network is also called *single layer with hidden units* or *network with one hidden layer*.

Fully Connected Layers

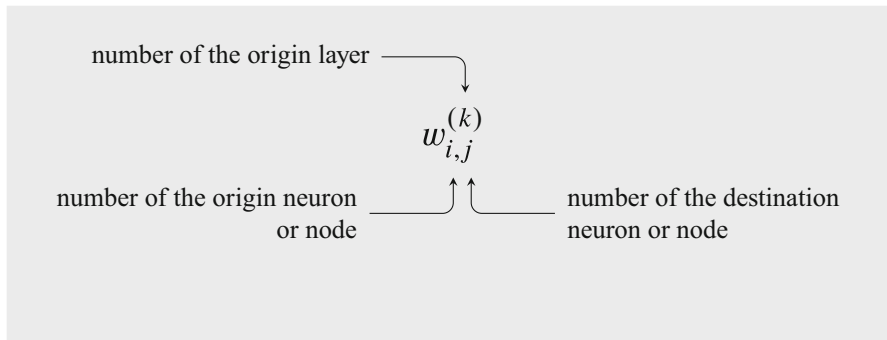
All nodes of the two neighboring layers are interconnected as shown in Fig. 17.12: each node on the origin layer (e.g. the input layer) is connected with all nodes of the destination layer (e.g. the hidden layer). Layers with this kind of connection are called *fully connected layers*. The two layers can have arbitrary numbers of nodes. For example, if the first layer has n nodes and the second layer has m nodes, then there are altogether $n \times m$ connections between the two layers.

Notations for Weights

Each of these connections is associated with a weight, where the weights between the first layer (the input layer) and the hidden layer are indicated by a superscript ⁽¹⁾, e.g., $w_{ij}^{(1)}$, and those between hidden and output layer are denoted as $w_{ij}^{(2)}$. To uniquely indicate the start and the end node of the sets of weights, we will use the following index notation:

Definition 17.1 Notation for Weights Each weight of the fully connected layers is uniquely characterized by one superscript and two subscripts:

(continued)



For example, the connection from input layer (layer no. 1) to hidden layer (layer no. 2) has $k = 1$ and the connection from the hidden to the output layer is $k = 2$. The subscripts in $w_{1,2}^{(2)}$ indicate that the first node/feature of the starting layer is connected to the second node or neuron of the receiving layer (starting and receiving are meant in terms of the information flow); a starting node with the number 0 indicates that this “weight” is in fact the bias, e.g., $w_{0,1}^{(1)}$ denotes the bias of the first neuron in the hidden layer.

Words of advice



The above introduced “index notation” should not be confused with a short notation for partial derivatives where, e.g., $f_{i,x} \equiv \partial f_i / \partial x$ indicates the partial derivative w.r.t. x of the i -th component of (the vector-valued) f .

Mathematical Formulation of the Network

We will now derive the governing equation for the first output node for the network shown in Fig. 17.12. For this we start from the output node and make our way through the hidden layer, back to the input layer. For simplicity, we only assume a single data record; the more general case of multiple data records that can be handled simultaneously will be formulated below in Sect. 17.4.5. We start our derivations from the first output y_1 . The corresponding output nodes simply sum up all incoming values together with a bias value $b_1^{(2)}$ (again, the bias is not shown in Fig. 17.12):

$$y_1 = b_1^{(2)} + w_{11}^{(2)} \xi_1 + w_{21}^{(2)} \xi_2 + w_{31}^{(2)} \xi_3 + w_{41}^{(2)} \xi_4 . \quad (17.43)$$

There, ξ_i is the output of the i -th neuron of the hidden layer and therefore, strictly speaking, also should have the superscript $^{(1)}$. If the bias is again treated as a weight, $w_{01}^{(2)}$, then Eq. 17.43 can be written as a scalar product of two vectors,

$$y_1 = \boldsymbol{\xi} \cdot \mathbf{w}_1^{(2)} \quad \text{with} \quad \boldsymbol{\xi} = [1, \xi_1, \dots, \xi_4]$$

$$\text{and } \mathbf{w}_1^{(2)} = [w_{01}^{(2)}, \dots, w_{41}^{(2)}]^\top. \quad (17.44)$$

The scalar product $\xi \cdot \mathbf{w}_1^{(2)}$ is shorter and looks more elegant than Eq. 17.43 but is also less explicit—in particular keep the “special” definition of ξ from Eq. 17.44 in mind and note that $\mathbf{w}_1^{(2)}$ is the vector that contains the weights for the output node number 2. The second output is computed in analogy, exchanging $\mathbf{w}_1^{(2)}$ with $\mathbf{w}_2^{(2)}$.

ξ_i is the output from neuron number i and can be computed by just following the same rule as we did already for the single Rosenblatt perceptron:

$$\begin{aligned} \xi_1 &= \varphi_1 \left(\mathbf{x} \cdot \mathbf{w}_1^{(1)} \right) \quad \text{with } \mathbf{x} = [1, x_1, x_2, x_3] \\ \text{and } \mathbf{w}_1^{(1)} &= [w_{01}^{(1)}, \dots, w_{41}^{(1)}]^\top. \end{aligned} \quad (17.45)$$

As before, we treat the bias as 0-th weight and therefore again extend the vectors $\mathbf{x}$ and $\mathbf{w}_1^{(1)}$ with 1 and $w_{01}^{(1)}$, respectively. The outputs from the other neurons are then computed in analogy, e.g.,

$$\xi_2 = \varphi_2 \left(\mathbf{x} \cdot \mathbf{w}_2^{(1)} \right). \quad (17.46)$$

17.4.5 “Vectorized” Operations: The Equations for the Network with a Hidden Layer

This can now be summarized for the network with three features and inputs, four units contained in the hidden layer, and two output nodes, starting from the final output of the network. Additionally, for the computational efficiency, it is beneficial to “vectorize” these steps. Vectorization implies that iterating over elements of a vector or matrix by using `for` loops can be avoided. Instead, by performing matrix–matrix or matrix–vector products, all calculations can be done virtually simultaneously because Python’s numpy library or MATLAB® is able to strongly optimize these operations.⁵ As compared to using regular `for` loops over individual elements of arrays, this may result in a speed up of the computational time by factors of 10 and even up to 1000 or more! Last but not least, for the current mathematical problem, such a notation additionally looks compact and thereby helps to understand the structure of the equations.

⁵ This is typically done by using pre-compiled and possibly strongly optimized code, which itself is typically written in a low-level language such as C or Fortran. *Vectorization* does not imply any parallel execution of code even though an already vectorized code often can easily be run on multi-cores/multi-processors as well.

$$\mathbf{y} = [y_1 \ y_2] = [1 \ \xi_1 \ \xi_2 \ \xi_3 \ \xi_4] \cdot \underbrace{\begin{bmatrix} w_{01}^{(2)} & w_{02}^{(2)} \\ w_{11}^{(2)} & w_{12}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} \\ w_{31}^{(2)} & w_{32}^{(2)} \\ w_{41}^{(2)} & w_{42}^{(2)} \end{bmatrix}}_{\mathbf{W}^{(2)}}. \quad (17.47)$$

Next, the intermediate values ξ_i can be computed by applying a different activation function to each of them:

$$\boldsymbol{\xi} = [\xi_1 \ \xi_2 \ \xi_3 \ \xi_4] = \left[\varphi_1(\mathbf{x} \cdot \mathbf{w}_1^{(1)}) \ \dots \ \varphi_4(\mathbf{x} \cdot \mathbf{w}_4^{(1)}) \right]. \quad (17.48)$$

Defining $\boldsymbol{\varphi}$ as the vector-valued function that consists of the φ_i , this can be written in short as

$$\boldsymbol{\xi} = \boldsymbol{\varphi} \left(\mathbf{x} \cdot \begin{bmatrix} w_{01}^{(1)} & w_{02}^{(1)} & w_{03}^{(1)} & w_{04}^{(1)} \\ w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} & w_{14}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} & w_{24}^{(1)} \end{bmatrix} \right) = \boldsymbol{\varphi}(\mathbf{x} \cdot \mathbf{W}^{(1)}), \quad (17.49)$$

where we additionally introduced the matrix of all weights between the input layer and the hidden layer as $\mathbf{W}^{(1)}$. These are now formulations that are straightforward to be generalized to a network with one hidden layer, an arbitrary number of features, arbitrary number of units contained in the hidden layer, and output nodes. Furthermore, without much of additional work, it is also possible to consider not just a single data record but *all* data records of the data matrix at once and in a vectorized manner.

We can now easily turn the vectorized equations into a Python implementation as shown in Python Listing 17.4.

17.4.6 Capabilities of the Networks for Classification and Regression

One of the most important components that determines whether the network performs a regression or a classification task is the output node. So far we encountered two different types of output nodes: the first one contained a threshold function—indicator for the fact that this is about classification. With a `True` or `False` (i.e., 1 or 0) output, we can perform a *binary classification* for deciding if something belongs to class A or to class B. The first more complex network, Fig. 17.12, contained two output nodes that were only performing (weighted) summations resulting in real numbers y_1 and y_2 and therefore is a regression model. This particular example takes three features as input, e.g., the spatial x , y , and z coordinates of a point and results in two real valued outputs, e.g., the

```

30 import numpy as np
31
32 """ == These are the used parameters and variables: =====
33 X: input data,          shape: (n_samples,      n_features)
34 W1: weights of hidden layer,  shape: (n_features + 1, n_units  )
35 W2: weights of output node,   shape: (n_units + 1, 1      )
36 theta1: threshold hidden layer, shape: (n_units)  -> a 1d array
37 theta1: threshold output node, shape: (1)        -> a 1d array
38 """
39
40 n_samples = X.shape[0]          # number of samples/data records
41
42 # prepend a "1"-column for bias of hidden layer neurons
43 X1 = np.concatenate((np.ones((n_samples, 1))), X), axis=1)
44
45 # copy the row vector theta 1 n_samples-times
46 THETA1 = np.repeat(theta1, n_samples, axis=0)
47 #
48 # evaluate the neurons of the hidden layer
49 T1 = np.where(X1 @ W1 >= THETA1, 1, -1)
50
51 # prepend a "1"-column for bias of output nodes
52 T1 = np.concatenate((np.ones((n_samples, 1))), T1), axis=1)
53
54 # compute results of output nodes
55 y = np.where(T1 @ W2 >= theta2, True, False)

```

Python Listing 17.4 General implementation of the vectorized equations for a network with one hidden layer. Note that `np.where` also works with the row vector `theta1`: it automatically applies it to each row of `X1 @ W1` so that creating `THETA1` is not required

temperature and a chemical concentration. Generalizing these approaches to, e.g., multiple classes and increasing their performance is a subject that will be covered at a later point after the training of these networks was introduced.

17.5 Summary and Historical Remarks

We saw how increasingly complex networks developed, starting from the example of the biological neuron as a “template.” All of the above developed networks of increasing complexity are able to represent increasingly complicated functions and, altogether, looked very promising in the 1960s. Problems intensively studied back then included the structure of “error surfaces” for Boolean operators of various complexities and how to represent assemblies of Boolean operators with possibly many inputs (see e.g. [6, 10] and references therein).

The greatest shortcoming, however, turned out to be a suitable “training algorithm” for more complex networks: transferring the strategy of the [ADALINE](#) training with the “delta rule” (or the Widrow & Hoff learning rule) from a single unit to more complex networks was not straightforward as the delta rule is based on the difference of the actual and desired output. The latter is available only for the unit that yields the output. Therefore, it is not clear how the weights and bias values of the other neurons in any of the hidden layers should be adjusted during the training.

This is why after a sustained period of excitement, [ANNs](#) was seen as a research “dead end” and, what is commonly known as the “AI winter” began and lasted until the end of the 1970s.

17.6 Exercises

17.1 Given are two Boolean variables x and y . Show how the Boolean operations AND, OR, and NOT can be expressed using “regular” number arithmetic, i.e., by interpreting `True` as 1 and `False` as 0.

17.2 Show that the perceptron is not able to represent the Boolean XOR operation.

17.3 Visualize a perceptron with two inputs that would be able to represent the logical AND operation. Also show the decision boundary in analogy to Fig. [17.4](#).

17.4 Assuming that we have Boolean functions with two input variables. Show that then there exist 16 different Boolean functions. Show geometrically that only two of them are not linearly separable

17.5 Assuming that we have Boolean functions with three input variables. How many different Boolean functions exist then and how many of them are linearly separable?

17.6 Visualize a perceptron with two inputs that would be able to represent the logical AND operation. Also show the decision boundary in analogy to Fig. [17.4](#).

17.7 Use the following function to create a dataset that consists of two normal distributions so that the data set has 2 features (the x and y coordinate) and the label 0 or 1.

```

1  def create_bivariate_normal_distribution(n_points=500):
2      X = np.empty((0, 2))
3      y = np.empty(0)
4
5      mean = [6, -2]
6      cov = [[1, -0.6], [-0.6, 3]]
7      x1, x2 = np.random.multivariate_normal(
8                  mean, cov, n_points
9              ).T
10     X = np.append(X, np.stack((x1, x2), axis=1),
11                  axis=0)
12     y = np.append(y, np.zeros_like(x1))
13
14     mean = [-1, 1]
15     cov = [[2, 0.6], [0.6, 12]]
16     x1, x2 = np.random.multivariate_normal(
17                 mean, cov, n_points
18             ).T

```

```

19 X = np.append(X, np.stack((x1, x2), axis=1),
20                  axis=0)
21 y = np.append(y, np.ones_like(x1))
22
23 return X, y

```

Use the Python Listing 17.2 to perform a classification. Create a plotting function that shows the decision boundary. Additionally, adjust the code such that the error is plotted as a function of the number of epoch.

17.8 The ADALINE model can also be used for regression problems. Which are the parts of the ADALINE model that need to be adapted? Apply your solution for predicting the inclination of the linear elastic regime of the MDS-1 (Tensile Test) dataset.

17.9 Assume that all weights and biases in a network of perceptrons are scaled by a constant value, $\lambda > 0$. How does this influence the network?

17.10 In Sect. 17.4.2 it was shown that for a particular set of weights, biases and thresholds a network of two TLUs is able to represent the Boolean operator XOR. The solution resulted in three regions for the output value y : two regions where $y = 1$ and one regions where $y = 0$.

- (i) Show by an example that there exists another class of solutions that consist of two regions where $y = 0$ and one regions where $y = 1$.
- (ii) Explain a systematic approach for finding these weights starting from the solution in Sect. 17.4.2.

References

1. G. Berlucchi and H. A. Buchtel. Neuronal plasticity: historical roots and evolution of meaning. *Exp Brain Res*, 192:307–319, 2009. DOI <https://doi.org/10.1007/s00221-008-1611-6>.
2. B. Blaus. Multipolar neuron. URL https://commons.wikimedia.org/wiki/File:Blausen_0657_MultipolarNeuron.png.
3. W. S. McCulloch and W. Pitts. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.
4. M. Minsky and S. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, Cambridge, MA, USA, 1969.
5. G. Nagy. Neural networks-then and now. *IEEE Transactions on Neural Networks*, 2(2):316–318, 1991. DOI <https://doi.org/10.1109/72.80343>.
6. R. Rojas. *Neural Networks—A Systematic Introduction*. Springer, Berlin, Heidelberg, 1st edition, 1996. ISBN 978-3-540-60505-8. DOI <https://doi.org/10.1007/978-3-642-61068-4>.
7. F. Rosenblatt. *The Perceptron—A Perceiving and Recognizing Automaton Project Para. Report: 85-460-1*, Cornell Aeronautical Laboratory. Cornell Aeronautical Laboratory, Buffalo, New York, Ithaca, New York, January 1957.
8. F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. DOI <https://doi.org/10.1037/h0042519>.

9. F. Rosenblatt. Principles of neurodynamics: Perceptrons and the theory of brain mechanisms, 1962.
10. Q. Sheng. *Threshold Logic*. Academic Press Inc, 1969. ISBN 978-0126398502.
11. W. Waldeyer. Ueber einige neuere Forschungen im Gebiete der Anatomie des Centralnervensystems). *DMW—Deutsche Medizinische Wochenschrift*, 17(44):1213–1218, Nov. 1891. DOI <https://doi.org/10.1055/s-0029-1206824>.
12. B. Widrow. An Adaptive “Adaline” Neuron Using Chemical “MEMISTORS”, Technical Report No. 15553-2, Stanford University, USA, 1960.